论文模板 04 · 高并发与高可用架构设计

> 配套素材见 `00-项目素材库.md`。本主题**几乎年年出现**，属必准备项。

一、真题常见问法

变体 A（高可用）

> 请论述你参与的某系统的高可用架构设计，包括冗余、故障转移、 容错与降级策略，并说明你如何验证架构的可用性。

变体 B（高性能）

> 请论述你如何通过架构手段提升系统的并发处理能力与响应性能。

变体 C（可靠性）

> 请论述你如何保障系统在异常场景下的数据可靠性与业务连续性。
>
> **判断**：BASE HOME 的强项在**变体 C（可靠性）**与**变体 A 的一部分**—— 因为项目有真实的重试、幂等、读写分离、会话外置等机制。 变体 B（性能）需要你补充真实压测数据。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的高可用架构设计与实现。
该平台服务日本 H.I.S. 集团营业本部与旅行者，业务具有强季节性——
长假前预约量激增、平峰期骤降，对系统的弹性与可靠性提出很高要求。
本人担任架构师，负责可用性设计、故障隔离方案与可靠性保障机制建设。
针对【在线业务与批量作业资源争抢】、【消息推送重复发送风险】、
【多实例部署下的会话一致性】三大挑战，
本文从【资源隔离】、【幂等与重试】、【状态外置】三个维度进行设计：
通过读写分离双数据源隔离批量作业影响，通过幂等时间窗与专用重试服务
杜绝重复推送，通过 Redis Cluster 实现会话外置与无状态扩容。
系统上线后【◆ 可用率/故障恢复时间】，验证了方案的可行性。

三、正文五段框架

第 1 段 · 项目背景与可用性目标（约 400 字）

必须写出「可用性目标」，这是高可用论文的标准起手式。

> 旅游业务对系统可用性有刚性要求：预约窗口期一旦系统不可用， 直接造成订单流失且无法补回。因此我们为该平台定义了明确目标： 🔶 年可用率目标（如 99.9%）、🔶 关键接口 P99 响应时间目标、 🔶 批处理作业完成时限（如次日营业开始前必须完成）。 这些目标驱动了后续所有的架构决策。

本项目背景：

> BASE HOME 采用 Java 21 + Spring Boot 3.5.7，部署于 AWS ap-northeast-1， 按多模块划分为 Web、Service、Batch、FaaS 等部署单元。 本人担任架构师，负责可用性设计与可靠性机制建设。

第 2 段 • 架构设计（约 800 字）

第一层：资源隔离——消除单点相互影响

> 可用性最大的敌人往往不是外部攻击，而是**内部资源争抢。** 本项目最典型的场景是：批处理需要从生产库大量读取并回写结果， 而在线预约接口同时也在访问同一数据库。 若共享连接池，夜间批处理峰值会直接拖慢甚至拖垮在线接口。
>
> 我的方案是为批处理配置**双数据源**： `ReaderDataSourceConfig`（`spring.datasource.reader`）与 `WriterDataSourceConfig`（`spring.datasource.writer`，`@Primary`）， 通过包路径隔离 Mapper（`repository/reader/` 与 `repository/writer/`）。 读写物理分离后，批量作业不再与在线业务争抢同一数据源连接。
>
> 同时设置 `TRANSACTION_READ_COMMITTED` 隔离级别， 减少长事务持有锁的时间； `defaultFetchSize = 200` 控制游标， 防止海量结果集一次性载入导致内存溢出——这是批处理最常见的 OOM 诱因。

第二层：故障隔离——按部署单元切分故障域

> Web、Service、Batch、FaaS 各自独立部署： - Batch 模块设置为 `web-application-type: none`，不占用 Web 端口， 与 Web 层无任何进程级耦合 - FaaS 侧 14 个 Lambda 按业务域拆分，**单个函数故障不会波及其他业务域** - 每个函数独立镜像、独立流水线、独立部署，支持单函数灰度与快速回滚
>
> **权衡说明**：这增加了运维单元数量与管理复杂度， 但把「一个模块崩溃导致全站不可用」的风险降为「单个业务域受影响」。

第三层：状态外置——支撑水平扩容

> Web 层多实例部署后，若 Session 存于本地内存， 负载均衡会把用户请求分发到不同实例，导致反复掉线。 我引入 **Spring Session Data Redis**，接入 **Redis Cluster（6 节点）**， 客户端使用 Jedis，会话超时设为 3600 秒。 会话外置后，Web 实例完全无状态，可任意水平扩容与重启而不影响在线用户。

第四层：数据层可靠性

> DynamoDB 侧启用乐观锁重试（`@EnableRetry`，重试 4 次、间隔 500ms），应对并发写入冲突；文件实体落 S3 而非数据库，降低库体积与备份恢复时长；主数据保留本地只读副本（26 张 `M*` 表），使外部 MDM 系统故障不影响业务查询。

第 3 段 • 关键技术实现（约 700 字）

要素一：幂等控制——防止重复推送

> 客户消息与邮件推送是本系统的可靠性敏感点：一旦重复发送，对客户是直接的体验事故。我们设计了多层防护： - **幂等时间窗**：`app.batch.idempotency-duration-minutes = 10`，同一业务键在窗口内重复触发会被拦截 - **专用重试服务**：`MailSendRetryService` 独立承担重试逻辑，与业务处理解耦，避免重试代码侵入业务流程 - **事务粒度拆分**：`service/transaction/*TransactionalService` 将事务边界控制在必要范围内，避免大事务回滚造成的连带重放 - **投递状态跟踪**：`TEmailDeliveryManagement` 表记录投递状态 与 `update_travel_status` 函数联动，形成可追溯的闭环

要素二：可观测性——故障可发现才可恢复

> 高可用的前提是「故障能被及时发现」。Batch 侧通过 **Prometheus PushGateway** 上报作业指标（配置 `shutdown-operation: post`，确保作业结束前完成指标推送），使批处理这类短生命周期进程的运行状态可被监控系统采集。FaaS 侧使用 `aws_lambda_powertools` 的 Logger 与 Tracer 输出结构化日志与链路追踪信息。

要素三：作业可重跑与可复现

> 批处理的重跑能力是可靠性的重要保障。我们将 `executionBaseDate`、`acquisitionRange`、`importTargetType` 等关键参数定义为 JobParameter 显式传入，不依赖系统时钟，使同一参数的重跑结果完全可复现。 🔶 补充：这解决了什么具体的补数场景。

第 4 段 • 难点与解决（约 700 字）

高可用主题最佳组合：

优先级	难点	适配理由
首选	⑦ 批量推送的幂等与重试	直接对应「可靠性保障」
次选	④ MySQL 读写分离双数据源	直接对应「资源隔离」
三选	⑨ 分布式会话 Redis Cluster	直接对应「无状态扩容」
加分	⑧ 跨国多时区一致性	对应「数据可靠性」，特色鲜明

难点示例（⑦，三段式写法）：

> **问题**：客户消息常需批量推送（`BulkPutCustomerMessage` 一次处理多条）， 邮件发送作业在遇到第三方网关限流或网络抖动时必然需要重试。 但朴素的重试会引入重复发送——对客户而言这是不可接受的体验事故。 这本质上是分布式环境下的**幂等性问题**，而非简单的异常处理问题。
>
> **方案**：我设计了三层防护。第一层是幂等时间窗， 以 10 分钟为窗口对同一业务键去重；第二层是把重试能力独立为 `MailSendRetryService`，与业务逻辑解耦， 使重试策略可独立调整而无需改动业务流程； 第三层是投递状态跟踪表，记录每一次投递的结果， 使异常场景下可以精确定位漏发与重发范围。
>
> **效果**：🔶 重复发送率降至 X%，🔶 作业失败可观测并支持定点重跑。

第 5 段 • 总结与不足（约 200 字）

目前该平台已稳定运行【🔶 时长】，年可用率达到【🔶 数值】，

批量作业在【🔶 时限】内稳定完成。

不足之处：

一是跨存储的最终一致性链路尚无统一的延迟监控，异常排查路径偏长；

二是 Lambda 冷启动在低频函数上仍有可感知延迟。

后续计划补充一致性延迟监控与预置并发配置。

通过本项目我认识到，高可用不是靠单一技术实现的，

而是由资源隔离、故障隔离、状态外置、幂等重试等一系列设计共同构成的体系。

四、本项目填充点速查

论文要素	直接可写的内容
资源隔离	Batch 读写分离双数据源，Mapper 按 `repository/reader` / `writer` 隔离
事务控制	`TRANSACTION_READ_COMMITTED`； `*TransactionalService` 拆分事务粒度
内存保护	`mybatis.defaultFetchSize = 200` 控制批量游标
故障隔离	Web/Service/Batch/FaaS 独立部署；14 个 Lambda 按业务域拆分
状态外置	Spring Session Data Redis + Redis Cluster（6 节点）+ Jedis，超时 3600s
并发控制	DynamoDB 乐观锁重试 4 次 / 500ms（`@EnableRetry`）
幂等	`app.batch.idempotency-duration-minutes = 10`
重试	`MailSendRetryService` 专用重试服务
投递跟踪	`TEmailDeliveryManagement` + `update_travel_status`
可观测	Prometheus PushGateway（Batch）；Powertools Logger/Tracer（FaaS）
可复现	JobParameter 显式传入 `executionBaseDate` / `acquisitionRange`
降级隔离	主数据本地只读副本（26 张 `M*` 表），外部系统故障不影响业务
传输安全	gRPC 使用 TLS（`tls://` endpoint）

五、可用难点素材（按适配度排序）

1. ⑦ **批量推送的幂等与重试** ☆☆☆ —— 本主题首选
2. ④ **MySQL 读写分离双数据源** ☆☆☆ —— 资源隔离
3. ⑨ **分布式会话 Redis Cluster** ☆☆☆ —— 无状态化
4. ⑧ **跨国多时区一致性** ☆☆☆ —— 数据可靠性（特色）
5. ⑥ **Serverless 数据同步** ☆☆ —— 最终一致性的可靠性设计

六、踩坑提醒

坑	后果	正确做法
只写「用了集群所以高可用」	空洞	要写**可用性目标 → 威胁 → 措施 → 验证**这条链
不提可用性指标	漏考点	必须给出 SLA 目标（◆ 填真实值）与 MTTR
编造双活/异地多活	高风险	本项目未见多活架构证据；写「多实例 + 状态外置 + 故障域隔离」是真实的
编造熔断限流框架	高风险	本项目**未见 Sentinel/Hystrix/Resilience4j 证据**，不要写「我们用了熔断降级」
编造 Redis 业务缓存	高风险	Redis 仅用于 Session，写成「多级缓存架构」会露馅
只讲技术不讲验证	漏考点	补上「如何验证」：TestContainers + LocalStack 做集成验证、Prometheus 做运行监控
忽略批量作业可靠性	片面	批处理是可靠性重点，务必写入幂等、重试、可重跑

> ⚠ **三条技术真实性红线**（务必遵守）： ① **未见熔断/限流组件**（Hystrix、Sentinel、Resilience4j） ② **未见 Redis 业务数据缓存**（仅 Session） ③ **未见多活/异地容灾架构**
>
> 若题目强制要求写熔断降级，可如实写： 「当前架构以故障域隔离与幂等重试为主要可靠性手段， 熔断降级尚未引入，这是后续改进方向」——**诚实反而加分**。

配套： `03-数据架构与混合存储.md` · `08-批处理架构设计.md` · `05-系统安全架构.md`